

Experiment: Deleted File Recovery using File Signatures

Aim

To scan a file for known file signatures (magic numbers) and identify embedded or recoverable file types using Python.

Algorithm (5 Points)

1. Create a dictionary containing common file signatures (magic numbers) and their corresponding file types.
2. Read the file path from the user and open the file in binary mode.
3. Read the entire contents of the file into memory.
4. Search for each file signature in the file data using the find() method.
5. Display the detected file type and its byte offset. If no signature is found, display an appropriate message.

Python Code

```
# deleted_file_recovery.py
```

```
signatures = {  
    b"%PDF": "PDF File",  
    b"\x89PNG\r\n\x1a\n": "PNG Image",  
    b"\xFF\xD8\xFF": "JPEG Image",  
    b"PK\x03\x04": "ZIP/DOCX/XLSX/PPTX File"  
}
```

```
filename = input("Enter file path to scan: ").strip().strip('"')
```

```
try:

    with open(filename, "rb") as file:

        data = file.read()


    found = False


    print("\nScanning for file signatures...\n")


    for signature, filetype in signatures.items():

        index = data.find(signature)


        if index != -1:

            print(f'{filetype} signature found at byte offset: {index}')

            found = True


    if not found:

        print("No recognizable file signatures found.")


except FileNotFoundError:

    print("File not found!")

except Exception as e:

    print("Error:", e)
```

Sample Input

Enter file path to scan:

C:\Users\saich\OneDrive\Desktop\Sai Charan Tej_Resume.pdf

Sample Output

Scanning for file signatures...

PDF File signature found at byte offset: 0

Result

The program successfully scanned the selected file and detected its file signature. It identified the file as a PDF File by locating the PDF magic number at byte offset 0, demonstrating how file signatures can be used in digital forensics for deleted file recovery and file type identification.